

Logistic Regression Analysis: Pima Indians Diabetes

Tilak Poudel MDS 3rd sem, Roll no. 36

2026-05-19

Contents

Load Libraries	1
1. Environment Setup & Data Loading	2
2. Data Cleaning & Imputation	2
3. Exploratory Data Analysis (EDA)	3
4. Data Splitting	4
5. Model Estimation & Diagnostics	5
Collinearity Testing	6
Odds Ratios:	6
6. Out-of-Sample Performance Evaluation	7
Confusion Matrix Diagnostics	7
Discrimination Mapping (ROC & AUC)	8
7. Real-Time Risk Prediction for Sample Patients	9

Load Libraries

```
library(mlbench)    # For PimaIndiansDiabetes2
library(tidyverse)  # For data transformation & plots
library(caret)      # For partitioning & confusion matrix
library(ROCR)       # For ROC & AUC calculation
library(mice)       # For handling missing values via imputation
library(car)        # For VIF calculations
```

1. Environment Setup & Data Loading

First, we load the required libraries for our analytical pipeline. We use `mlbench` for our source data, `mice` for handling missing features, `tidyverse` for data manipulation/visualization, and `caret`/`ROCR` for cross-validation diagnostics.

```
data("PimaIndiansDiabetes2")
glimpse(PimaIndiansDiabetes2)
```

```
#> Rows: 768
#> Columns: 9
#> $ pregnant <dbl> 6, 1, 8, 1, 0, 5, 3, 10, 2, 8, 4, 10, 10, 1, 5, 7, 0, 7, 1, 1~
#> $ glucose <dbl> 148, 85, 183, 89, 137, 116, 78, 115, 197, 125, 110, 168, 139, ~
#> $ pressure <dbl> 72, 66, 64, 66, 40, 74, 50, NA, 70, 96, 92, 74, 80, 60, 72, N~
#> $ triceps <dbl> 35, 29, NA, 23, 35, NA, 32, NA, 45, NA, NA, NA, NA, 23, 19, N~
#> $ insulin <dbl> NA, NA, NA, 94, 168, NA, 88, NA, 543, NA, NA, NA, NA, 846, 17~
#> $ mass <dbl> 33.6, 26.6, 23.3, 28.1, 43.1, 25.6, 31.0, 35.3, 30.5, NA, 37.~
#> $ pedigree <dbl> 0.627, 0.351, 0.672, 0.167, 2.288, 0.201, 0.248, 0.134, 0.158~
#> $ age <dbl> 50, 31, 32, 21, 33, 30, 26, 29, 53, 54, 30, 34, 57, 59, 51, 3~
#> $ diabetes <fct> pos, neg, pos, neg, pos, neg, pos, neg, pos, pos, neg, pos, n~
```

```
summary(PimaIndiansDiabetes2)
```

```
#>      pregnant      glucose      pressure      triceps
#> Min.   : 0.000   Min.   : 44.0   Min.   : 24.00   Min.   : 7.00
#> 1st Qu.: 1.000   1st Qu.: 99.0   1st Qu.: 64.00   1st Qu.:22.00
#> Median : 3.000   Median :117.0   Median : 72.00   Median :29.00
#> Mean   : 3.845   Mean   :121.7   Mean   : 72.41   Mean   :29.15
#> 3rd Qu.: 6.000   3rd Qu.:141.0   3rd Qu.: 80.00   3rd Qu.:36.00
#> Max.   :17.000   Max.   :199.0   Max.   :122.00   Max.   :99.00
#>      NAs      :5      NAs      :35      NAs      :227
#>      insulin      mass      pedigree      age      diabetes
#> Min.   : 14.00   Min.   :18.20   Min.   :0.0780   Min.   :21.00   neg:500
#> 1st Qu.: 76.25   1st Qu.:27.50   1st Qu.:0.2437   1st Qu.:24.00   pos:268
#> Median :125.00   Median :32.30   Median :0.3725   Median :29.00
#> Mean   :155.55   Mean   :32.46   Mean   :0.4719   Mean   :33.24
#> 3rd Qu.:190.00   3rd Qu.:36.60   3rd Qu.:0.6262   3rd Qu.:41.00
#> Max.   :846.00   Max.   :67.10   Max.   :2.4200   Max.   :81.00
#>      NAs      :374      NAs      :11
```

2. Data Cleaning & Imputation

```
colSums(is.na(PimaIndiansDiabetes2))
```

```
#> pregnant glucose pressure triceps insulin mass pedigree age
#>         0         5         35         227         374         11         0         0
#> diabetes
#>         0
```

To avoid discarding partial information, we execute Multiple Imputation by Chained Equations (MICE) using Predictive Mean Matching (PMM) to fill missing data points based on surrounding patient trends.

```
set.seed(123)
# Run imputation routing
imputed_data <- mice(PimaIndiansDiabetes2, m = 1, method = "pmm", printFlag = FALSE)

# Build out the completed dataset
clean_diabetes <- complete(imputed_data)

sum(is.na(clean_diabetes))
```

```
#> [1] 0
```

3. Exploratory Data Analysis (EDA)

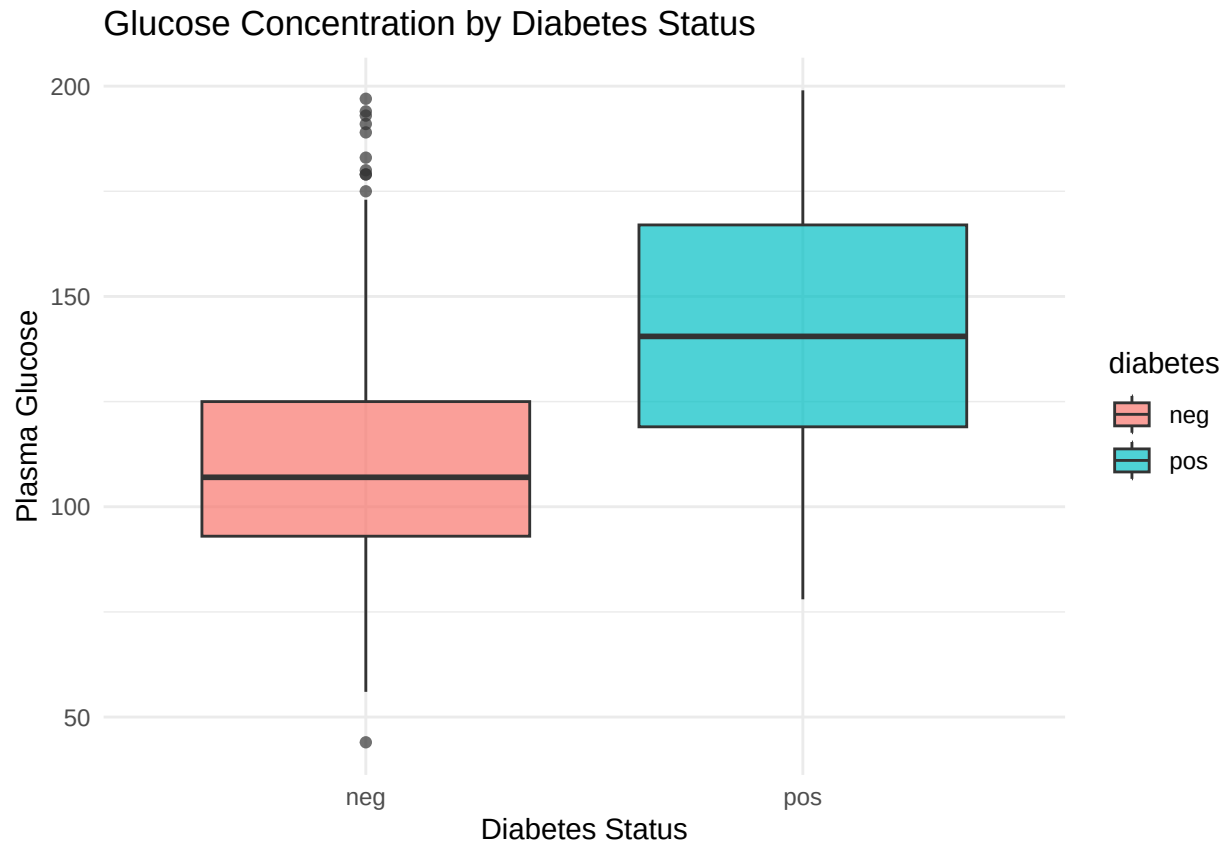
Before building our statistical equations, we check the baseline distribution of the target variable (diabetes) to establish class balances.

```
prop.table(table(clean_diabetes$diabetes))
```

```
#>
#>      neg      pos
#> 0.6510417 0.3489583
```

We now isolate plasma glucose values across positive and negative diagnostic states to visualize group variance.

```
ggplot(clean_diabetes, aes(x = diabetes, y = glucose, fill = diabetes)) +
  geom_boxplot(alpha = 0.7) +
  theme_minimal() +
  labs(
    title = "Glucose Concentration by Diabetes Status",
    x = "Diabetes Status",
    y = "Plasma Glucose"
  )
```



The boxplot visualizing plasma glucose concentration against diabetes status demonstrates a clear, foundational biological relationship: individuals diagnosed with diabetes typically exhibit significantly higher blood glucose levels compared to those without the condition.

Observation:

The graph provides visual proof that **glucose** is a highly significant differentiator, while simultaneously showing enough variance and overlap to confirm that a multi-variable statistical model is necessary to make accurate predictions.

4. Data Splitting

We perform an 80/20 train-test split using stratified sampling via caret to ensure equivalent base-rate metrics in both validation matrices.

```
set.seed(42)

# Generate partition indexes
train_index <- createDataPartition(clean_diabetes$diabetes, p = 0.8, list = FALSE)

train_set <- clean_diabetes[train_index, ]
test_set <- clean_diabetes[-train_index, ]
```

5. Model Estimation & Diagnostics

We construct a binary generalized linear model using the complete matrix of explanatory features under a logit link function.

```
diabetes_model <- glm(diabetes ~ ., data = train_set, family = binomial)
summary(diabetes_model)
```

```
#>
#> Call:
#> glm(formula = diabetes ~ ., family = binomial, data = train_set)
#>
#> Coefficients:
#>              Estimate Std. Error z value Pr(>|z|)
#> (Intercept) -9.4588942  0.9397847 -10.065  < 2e-16 ***
#> pregnant      0.1652091  0.0366958   4.502 6.73e-06 ***
#> glucose       0.0407534  0.0049334   8.261  < 2e-16 ***
#> pressure     -0.0045708  0.0095603  -0.478   0.633
#> triceps       0.0044097  0.0134558   0.328   0.743
#> insulin      -0.0008093  0.0011466  -0.706   0.480
#> mass          0.0871179  0.0219512   3.969 7.23e-05 ***
#> pedigree      1.0301626  0.3333315   3.091   0.002 **
#> age           0.0001159  0.0110795   0.010   0.992
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#>      Null deviance: 796.05  on 614  degrees of freedom
#> Residual deviance: 557.13  on 606  degrees of freedom
#> AIC: 575.13
#>
#> Number of Fisher Scoring iterations: 5
```

Observation

Statistical Significance: Who Matters? Let's look at the $\text{Pr}(>|z|)$ column (the p-values) and the asterisks (***). If a p-value is less than 0.05, that variable is a statistically significant predictor of diabetes.

Highly Significant Predictors (*** or):

- glucose ($p < 2 \times 10^{-16}$) — By far the strongest predictor.
- pregnant ($p = 6.73 \times 10^{-6}$)
- mass (BMI) ($p = 7.23 \times 10^{-5}$)
- pedigree (Diabetes pedigree function) ($p = 0.002$)

Not Significant Predictors:

- pressure ($p = 0.633$),
- triceps ($p = 0.743$),
- insulin ($p = 0.480$), and
- age ($p = 0.992$).

Insight: Notice that age is not significant here. That doesn't mean age doesn't matter in real life; it means that once we already know a patient's glucose, BMI, and pregnancy history, adding their exact age provides no additional predictive power to the model.

Collinearity Testing

We calculate the Variance Inflation Factor (VIF) to ensure that inter-variable correlations do not warp our standard error values.

```
vif(diabetes_model)

#> pregnant  glucose pressure  triceps  insulin    mass pedigree    age
#> 1.481160 1.464261 1.237956 1.625870 1.463801 1.816190 1.015777 1.629219
```

Observation:

No VIF metrics cross above the conservative benchmark threshold ($VIF > 5$). Structural multicollinearity is not an issue in this context.

Odds Ratios:

To evaluate effect sizing intuitively, we convert log-odds metrics to standard risk-based Odds Ratios (OR) alongside their 95% confidence bounds.

```
exp(cbind(OR = coef(diabetes_model), confint(diabetes_model)))

#>                OR          2.5 %          97.5 %
#> (Intercept) 7.799278e-05 1.151529e-05 0.0004612944
#> pregnant    1.179640e+00 1.098953e+00 1.2693018165
#> glucose     1.041595e+00 1.031896e+00 1.0520875367
#> pressure    9.954396e-01 9.769042e-01 1.0142894378
#> triceps     1.004419e+00 9.777745e-01 1.0309644392
#> insulin     9.991910e-01 9.969476e-01 1.0014600304
#> mass        1.091025e+00 1.045862e+00 1.1400777567
#> pedigree    2.801521e+00 1.467911e+00 5.4339688679
#> age         1.000116e+00 9.783402e-01 1.0218980808
```

Observation:

An Odds Ratio tells how the likelihood of the outcome (having diabetes) changes with a 1-unit increase in that specific predictor variable.

- $OR = 1.0$: The predictor has no effect on the odds of diabetes.
- $OR > 1.0$: The predictor increases the odds of diabetes (risk factor).
- $OR < 1.0$: The predictor decreases the odds of diabetes (protective factor).

Glucose ($OR = 1.0416$):

For every 1-unit increase in plasma glucose concentration, the odds of a patient developing diabetes increase by 4.16% (calculated as $1.0416 - 1 = 0.0416$).

Mass / BMI (OR = 1.0910):

For every 1-unit increase in BMI, a patient's odds of being diabetic increase by 9.1% ($1.0910 - 1$). If someone's BMI jumps by 10 units, their risk multiplies significantly.

6. Out-of-Sample Performance Evaluation

Now we apply our fitted parameters to the untouched validation test set. Predictions are generated as probabilities and converted to factor levels using the traditional 0.5 classification cutoff threshold.

```
# Calculate continuous risk probabilities
test_set$prob <- predict(diabetes_model, newdata = test_set, type = "response")

# Convert probabilities to factors
test_set$pred_class <- factor(ifelse(test_set$prob > 0.5, "pos", "neg"), levels = c("neg", "pos"))
```

Confusion Matrix Diagnostics

We calculate predictive indicators (Accuracy, Sensitivity, and Specificity) to evaluate spatial performance.

```
confusionMatrix(test_set$pred_class, test_set$diabetes, positive = "pos")
```

```
#> Confusion Matrix and Statistics
#>
#>           Reference
#> Prediction neg pos
#>      neg   87  23
#>      pos   13  30
#>
#>              Accuracy : 0.7647
#>              95% CI   : (0.6894, 0.8294)
#>    No Information Rate : 0.6536
#>    P-Value [Acc > NIR] : 0.001988
#>
#>              Kappa   : 0.4563
#>
#>  Mcnemar's Test P-Value : 0.133614
#>
#>      Sensitivity : 0.5660
#>      Specificity : 0.8700
#>    Pos Pred Value : 0.6977
#>    Neg Pred Value : 0.7909
#>      Prevalence   : 0.3464
#>    Detection Rate : 0.1961
#>  Detection Prevalence : 0.2810
#>    Balanced Accuracy : 0.7180
#>
#>      'Positive' Class : pos
#>
```

Observation:

Accuracy : 0.7647

- True Negatives (87): The model correctly predicted 87 healthy patients as neg.
- True Positives (30): The model correctly predicted 30 diabetic patients as pos.
- False Positives (13): The model flagged 13 healthy people as diabetic.
- False Negatives (23): The model cleared 23 diabetic patients, telling them they were healthy.

Sensitivity (0.5660):

Out of all the people who actually have diabetes in test pool, the model successfully caught 56.6% of them.

Critique: This is the weakest link in the current model. It means we are missing 43.4% of the diabetic patients (the 23 False Negatives), sending them home without treatment.

Specificity (0.8700):

Out of all the truly healthy people, the model correctly identified 87.0% of them. It is highly specific, meaning it rarely panics or triggers false alarms for healthy individuals.

Discrimination Mapping (ROC & AUC)

Finally, we run a threshold sweep across the true positive/false positive spectrum to establish discrimination power via the Area Under the Curve (AUC).

```
# 1. Generate performance tracking profiles
pred_obj <- prediction(test_set$prob, test_set$diabetes)
perf_obj <- performance(pred_obj, measure = "tpr", x.measure = "fpr")

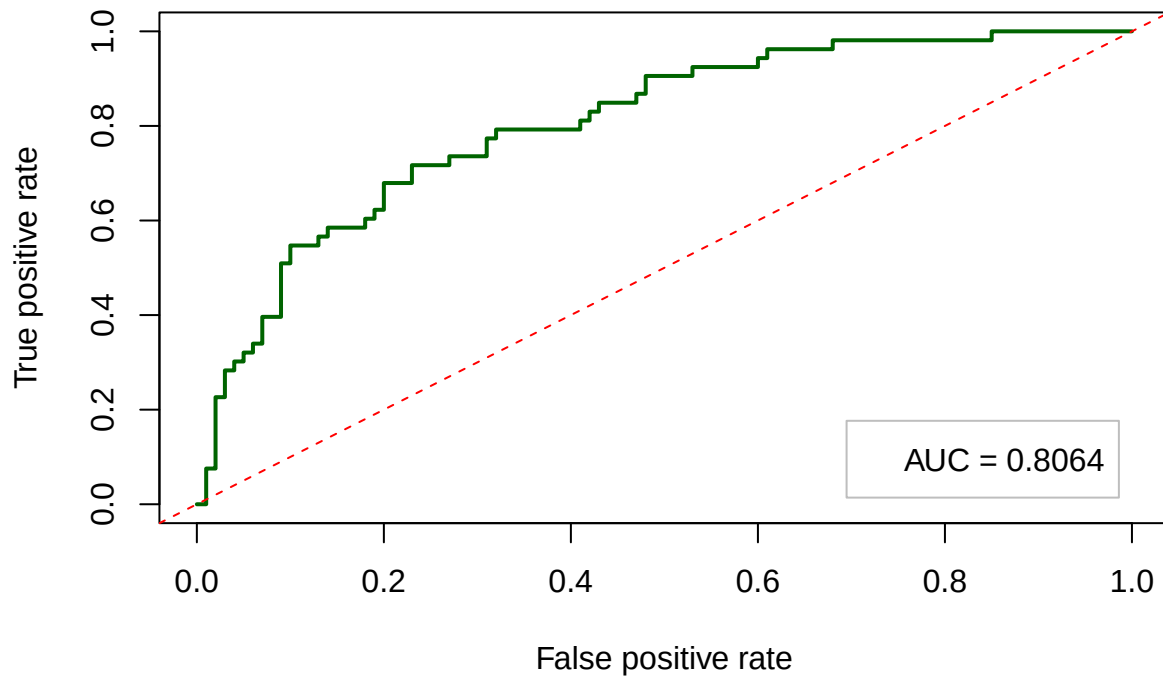
# 2. Extract numeric AUC value dynamically
auc_obj <- performance(pred_obj, measure = "auc")
auc_value <- round(auc_obj@y.values[[1]], 4)
cat("Final Area Under the Curve (AUC):", auc_value, "\n")
```

```
#> Final Area Under the Curve (AUC): 0.8064
```

```
# 3. Plot the base ROC Curve Space
plot(perf_obj, col = "darkgreen", lwd = 2, main = "ROC Curve: Pima Indians Diabetes Model")
abline(a = 0, b = 1, lty = 2, col = "red") # Random baseline

# 4. Corrected Legend Overlay
legend("bottomright",
      legend = paste("AUC =", auc_value),
      bty = "o", # Draws a complete border box ("o" for outer box)
      box.col = "gray", # Border color
      bg = "white", # Box background fill
      inset = 0.05) # Slight padding away from the exact margins
```


ROC Curve: Pima Indians Diabetes Model



7. Real-Time Risk Prediction for Sample Patients

We pass completely new, hypothetical patient observations through our model matrix to observe how risk calculations change based on clinical traits.

```
# Create custom sample patients matching the original data framework
new_patients <- tibble(
  pregnant = c(6, 1),
  glucose = c(165, 95),
  pressure = c(82, 70),
  triceps = c(35, 18),
  insulin = c(180, 60),
  mass = c(38.5, 22.0),
  pedigree = c(1.25, 0.25),
  age = c(45, 24)
)

# Assign readable profile names
rownames(new_patients) <- c("Patient_1_HighRisk", "Patient_2_LowRisk")

# Predict exact continuous probabilities
new_patients$Predicted_Probability <- predict(diabetes_model, newdata = new_patients, type = "response")

# Convert to final binary decision tag (using our standard 0.5 threshold)
```

```

new_patients$Predicted_Class <- ifelse(new_patients$Predicted_Probability > 0.5, "Positive", "Negative")

# Display the prediction matrix
knitr::kable(new_patients %>% select(glucose, mass, pedigree, Predicted_Probability, Predicted_Class),
              caption = "Model Assessment of New Patient Samples")

```

Table 1: Model Assessment of New Patient Samples

glucose	mass	pedigree	Predicted_Probability	Predicted_Class
165	38.5	1.25	0.9267278	Positive
95	22.0	0.25	0.0283508	Negative